Creative Software Design
7. Lab 1
STL – Standard Template Library

Dept. of Computer Science

Hanyang University

Template

- For same logic structure but different data types
  - Upon declaration, the compiler generates complete code.
- Both functions and classes can be **templated**

```
template <typename T>
T max(T a, T b) {
  return a > b ? a : b;
}
```

```
template <typename T>
class Point {
  T x, y;
public:
  Point(T a, T b) { x = a; y = b; }
  void move(T a, T b) {
    x += a; y += b;
  }
};
```

Standard Template Library

- Flexible and reusable with ***template***
- Robust and efficient libraries used in many fields
- We've checked in the class
  - ***Containers*** to store data of any types
  - ***Iterators***: used to manipulate container elements
  - ***Algorithms***: operations on containers for searching, sorting and many others

`std::vector` vs. `std::list`

- Both are sequence container classes
  - Manages stored data and its order (i.e., sequence)
- But implementations are different
  - `std::vector` implemented as a dynamic array
  - `std::list` implemented as a doubly-linked list
- Both Provide ***iterators*** to traverse elements in sequence
  - `std::vector` can iterates over the iterator
  - `std::vector` still can iterates as a array
- Two are different characteristics on
  - Random access
  - Insert or delete on a certain position

#1 – STL Relies on Templates

- Complete 3 functions to print the given array
  - Write three different functions
  - Write a single function template

```
#include <iostream>

int main(void) {
  int iarr[] = {10, 3, 8, 1, 2};
  int darr[] = {3.5, 31.4, -3.2, 10.15};
  std::string names = {"Alice", "bob", "Chalie",
    "David", "Ethan"};

  print(iarr, 5);
  print(darr, 4);
  print(names, 3);

  return 0;
}
```

#2 – Class Templates

- Complete 3 classes to contain and print the given array
  - Write three different classes and member functions
  - Write a single class template

```
#include <iostream>

int main(void) {
  int iarr[] = {10, 3, 8, 1, 2};
  int darr[] = {3.5, 31.4, -3.2, 10.15};
  std::string names = {"Alice", "bob", "Chalie",
    "David", "Ethan"};

  IArrayContainer icon;
  DArrayContainer dcon;
  NArrayContainer ncon;

  icon.Print();
  dcon.Print();
  ncon.Print();

  return 0;
}
```

#3 – STL Containers: `vector` & `list`

- Store score records from up to 10 students
- Input commands: new, print, quit
  - new: new student
  - print: print all scores of students
  - quit: quit

```
Commands(new, print, quit): n
2026020789
Alice
3.5
Commands(new, print, quit): n
2026020790
Bob
4.2
Commands(new, print, quit): p
[0]: 2026020789, Alice, 3.5
[1]: 2026020790, Bob, 4.2
Commands(new, print, quit): q
```

#4 – Find Min/Max Values

- Based on `example01.cpp`
- Find both min and max values from the given array

```
ForwardIterator min_element(ForwardIterator first, ForwardIterator last);
ForwardIterator min_element(ForwardIterator first, ForwardIterator last, Compare comp);
```

```
#include <iostream>

int main(void) {
  std::vector<int> iarr[] = {10, 3, 8, 1, 2};
  std::list<double> darr[] = {3.5, 31.4, -3.2, 10.15};



  return 0;
}
```

#5 – sort() & binary_search()

- Based on `example01.cpp`
- Find the given value from the array

```
#include <iostream>

int main(void) {
  std::vector<int> iarr[] = {10, 3, 8, 1, 2};
  std::list<double> darr[] = {3.5, 31.4, -3.2, 10.15};

  // find 3 from iarr
  // find 31.4 from darr

  return 0;
}
```

All Done!!

- Let's submit to ***gitlab***
- Push to ***gitlab*** after each lab session
  - At your VM instance (and make sure **pwd** at "week07_1")

```
# You are at {Your local project directory}/ week07_1
$ git add Makefile *.cpp *.h
$ git commit –m "submission of week07_1"
$ git push origin main
```